Cum funcționează PrefixSpan în acest proiect

Documentul parcurge:

1. Problema de sequential pattern mining
2. PrefixSpan clasic (Pei et al., 2001) — ideea, pseudocodul, un mic trace
3. Ce face de fapt biblioteca `prefixspan` din Python (și dacă e clasică sau nu)
4. Un exemplu lucrat pe cinci „pacienți" de jucărie, trecut prin fiecare îmbunătățire pe care am adăugat-o (v0 → v5), arătând ce schimbă fiecare layer și de ce contează

Citește-l liniar. Secțiunea 2 îți dă algoritmul. Secțiunea 4 îți dă engineering-ul.

Folosim **același** running example peste tot: cinci pacienți, cu items prefixați după tip ca să se vadă logica de diversificare. Îi introducem mai jos și îi păstrăm până la final.

---

0. Running example și encoding

Cei cinci pacienți (un timeline = o `sequence` ordonată cronologic de clinical events):

```
P1  <C:diabetes, M:metformin, P:foot_exam, C:hypertension>
P2  <C:diabetes, M:metformin, P:foot_exam>
P3  <C:diabetes, M:metformin, M:medication_review_due>
P4  <M:metformin, C:diabetes, P:foot_exam>
P5  <C:asthma, M:albuterol, M:medication_review_due, M:metformin>
```

Biblioteca `prefixspan` **nu** lucrează pe stringuri — lucrează pe întregi. Exact asta face funcția `encode()` din cod: parcurge toate sequences și mapează fiecare item distinct la un întreg unic, în **ordinea primei apariții**. Pe exemplul nostru, parcurgând P1 → P5, vocabularul iese:

```
C:diabetes              -> 0
M:metformin             -> 1
P:foot_exam             -> 2
C:hypertension          -> 3
M:medication_review_due -> 4
C:asthma                -> 5
M:albuterol             -> 6
```

iar sequences encodate (adică exact lista de liste pe care o primește `PrefixSpan`) sunt:

```
P1  [0, 1, 2, 3]
P2  [0, 1, 2]
P3  [0, 1, 4]
```

```
P4  [1, 0, 2]
P5  [5, 6, 4, 1]
```

Deci input-ul real al algoritmului e `[[0,1,2,3], [0,1,2], [0,1,4], [1,0,2], [5,6,4,1]]`. La final, `inv_vocab` (lista inversă întreg → string) traduce pattern-urile găsite înapoi în text citibil. Tot trace-ul din §2 e făcut pe acești întregi; păstrăm și echivalentul-string în paranteze ca să fie ușor de urmărit.

---

1. Problema

Avem o **sequence database**: o listă de sequences, fiecare fiind o listă ordonată de items. Pe exemplul nostru (encodat):

```
SID  Sequence
P1   [0, 1, 2, 3]
P2   [0, 1, 2]
P3   [0, 1, 4]
P4   [1, 0, 2]
P5   [5, 6, 4, 1]
```

Un **sequential pattern** e o subsequence (items în ordine, cu goluri permise) care apare în cel puțin `minsup` sequences. **Support**-ul unui pattern e numărul de sequences care îl conțin.

Pentru `minsup = 3`, pattern-ul `[0, 1]` (`C:diabetes, M:metformin`) e frequent: apare în P1, P2, P3 (support = 3). `[1, 0]` (`M:metformin` apoi `C:diabetes`) nu e — apare doar în P4.

Scopul: găsește toate frequent patterns, sau top-K după support.

Abordarea naivă (GSP / Apriori-style) generează length-2 candidates, scanează DB-ul, generează length-3 candidates, scanează din nou etc. Cu un vocabular de *V* items și patterns până la length *L*, poți ajunge să verifici O(V^L) candidates. Asta explodează rapid pe date clinice (V e în ordinul miilor).

---

2. PrefixSpan clasic

Ideea cheie (Pei, Han, et al., 2001): în loc să generezi candidates și să le testezi, crești patterns adăugând câte un item la un **prefix** și recursând pe o **projected database** care conține doar „ce vine după prefix" în fiecare sequence. Asta e *pattern-growth*, nu candidate-generation.

2.1 Projection

Prefix-projection-ul unui database `D` față de un prefix `<p>` e:

> Pentru fiecare sequence din `D` care conține `<p>`: ia suffix-ul imediat după prima apariție a lui `<p>`.

Sequences care nu conțin `<p>` sunt eliminate.

Exemplu: proiectăm P1..P5 pe prefix `[0]` (`C:diabetes`):

```
P1: [0, 1, 2, 3]  ->  [1, 2, 3]
P2: [0, 1, 2]     ->  [1, 2]
P3: [0, 1, 4]     ->  [1, 4]
P4: [1, 0, 2]     ->  [2]          (suffix-ul după primul 0)
P5: [5, 6, 4, 1]  ->  (eliminat, nu conține 0)
```

Acum, ca să găsești frequent patterns care încep cu `[0]`, scanezi doar această projected DB.

2.2 Algoritmul

```
PrefixSpan(prefix, projDB, minsup):
    # Găsește fiecare item care e frequent în projDB
    counts = {}
    for suffix in projDB:
        for item in unique(suffix):           # numără item-ul o dată per sequence
            counts[item] += 1

    for item, sup in counts.items():
        if sup >= minsup:
            newPrefix = prefix + [item]
            output(newPrefix, sup)
            newProjDB = project(projDB, item) # suffix după primul 'item' în
fiecare
            PrefixSpan(newPrefix, newProjDB, minsup)
```

Pornești cu `PrefixSpan([], D, minsup)`.

Ăsta e tot algoritmul. Arborele de recursie e exact mulțimea de frequent patterns.

2.3 Trace mic

Database-ul (P1..P5 de mai sus, encodat), `minsup = 3`.

Runda 1 — `prefix = []`, scanăm tot DB-ul:

```
item                     sequences care îl conțin   support
0  (C:diabetes)          P1, P2, P3, P4             4
1  (M:metformin)         P1, P2, P3, P4, P5         5
2  (P:foot_exam)         P1, P2, P4                 3
3  (C:hypertension)      P1                         1   -> drop
4  (M:medication_review) P3, P5                     2   -> drop
5  (C:asthma)            P5                         1   -> drop
6  (M:albuterol)         P5                         1   -> drop
```

Frequent length-1: `[0]:4`, `[1]:5`, `[2]:3`.

Recursăm pe `[0]`. Proiectăm (arătat în 2.1):

```
projDB[0] = [[1,2,3], [1,2], [1,4], [2]]
```

Numărăm items în projection (o dată per sequence):

```
1: 3 (în [1,2,3], [1,2], [1,4])   -> frequent
2: 3 (în [1,2,3], [1,2], [2])     -> frequent
4: 1                              -> drop
3: 1                              -> drop
```

Output `[0,1]:3` (`C:diabetes, M:metformin`), `[0,2]:3` (`C:diabetes, P:foot_exam`). Recursăm pe fiecare.

Recursăm pe `[0,1]`. Proiectăm projection-ul pe `1`:

```
[1,2,3] -> [2,3]
[1,2]   -> [2]
[1,4]   -> [4]
[2]     -> (eliminat, nu conține 1)
```

Counts: `2:2, 4:1`. Niciun item nu atinge support 3 → backtrack. Deci `[0,1,2]` are doar support 2 și **nu** e frequent la `minsup=3`.

> Observă de ce: în P4 `[1,0,2]`, `M:metformin` vine *înaintea* lui `C:diabetes`, deci P4 nu conține pattern-ul `0 → 1`. Asta e exact problema pe care o repară tie-break sort-ul din v1 (§4).

Recursăm pe `[0,2]`. Proiectăm projDB[0] pe `2`:

```
[1,2,3] -> [3]
[1,2]   -> []
[1,4]   -> (eliminat)
[2]     -> []
```

Counts: `3:1` → drop. Backtrack.

Similar recursăm pe `[1]`, apoi pe `[2]`:

- `[1]` → projDB[1] dă `[2]:3` (P1, P2, P4) → output `[1,2]:3` (`M:metformin, P:foot_exam`); extinderea nu mai atinge support 3.
- `[2]` → projDB[2] = `[[3],[],[]]`, nimic frequent.

Mulțimea completă de patterns cu length≥2 la `minsup=3` e `{[0,1]:3, [0,2]:3, [1,2]:3}`. Gata.

De ce e rapid: la fiecare nivel scanezi doar projection-ul, care se micșorează rapid. Nu enumeri niciodată candidates care nu apar de fapt.

2.4 Optimizarea pseudo-projection

A materializa fiecare projected DB ca liste noi e risipitor. Lucrarea introduce **pseudo-projection**: fiecare „matched sequence" din projection e stocată ca un pointer `(sequence_id, last_match_position)` în DB-ul original. Ca să extinzi prefix-ul, parcurgi sequence-ul original de la `last_match_position + 1` și cauți următorul item.

E o optimizare de memorie și de timp de copiere, dar algoritmul — recursie pe prefix tree, projected counting la fiecare nod — rămâne neschimbat.

---

3. Ce face biblioteca

Pachetul `prefixspan` de pe PyPI implementează **PrefixSpan clasic cu pseudo-projection**. Uită-te în `prefixspan/topk.py` din venv-ul tău:

```
def topk_rec(patt, matches):
    if len(patt) >= self.minlen:
        verify(patt, matches)
        if len(patt) == self.maxlen:
            return
    occurs = nextentries(self._db, matches)   # construiește projected counts
    for newitem, newmatches in sorted(occurs.items(), ...):
        newpatt = patt + [newitem]
        ...
        topk_rec(newpatt, newmatches)         # recursie
```

- `matches` e o listă de tuple `(sequence_index, last_match_position)` — pseudo-projection.
- `nextentries` (în `extratools.dicttools`) construiește un inverted index `{item: matches_for_that_item}` din suffix-urile `data[i][lastpos+1:]`. Asta e exact pasul „project then count items" din §2.2.
- Recursia e prefix-growth tree-ul.

Ce adaugă biblioteca peste algoritmul clasic:

- **Top-K printr-un heap**: oprire timpurie când nicio extindere nu poate bate al K-lea cel mai bun support (verificarea `canpass`).
- **`closed=True`**: prunează supersets cu support identic și emite doar closed patterns (`isclosed`, `canclosedprune` în `closed.py`).
- **`generator=True`**: noțiunea duală de minimal-generator patterns.

Deci: da, e algoritmul clasic. „Optimizările" pe care le expune biblioteca sunt top-K early termination și closed/generator post-filters — nu un algoritm diferit.

---

4. Aceleași cinci sequences prin fiecare layer

Folosim aceeași sequence database (cei cinci pacienți de mai sus).

`minsup_pct = 0.40` → `abs_minsup = 2` (40% din 5). `TOP_K = 3` pentru exemplu (ca să putem arăta comportamentul de capping).

> Notă: în §2 am folosit `minsup=3` pentru un trace curat; aici, în §4, folosim `abs_minsup=2` (pragul real al codului la 40%), deci unele patterns suplimentare devin frequent.

v0 — PrefixSpan brut

```
sequences = build_sequences(engine)            # grupare cronologică
PrefixSpan(encode(sequences)).topk(3)
```

Ce găsește (top-3 după support):

```
1. [1]        (M:metformin)              sup=5   (toți)
2. [0]        (C:diabetes)               sup=4
3. [0, 1]     (C:diabetes, M:metformin)  sup=3   (P1,P2,P3)
```

Deja o problemă: length-1 patterns domină, iar `medication_review_due` (un noise item cunoscut) ar ieși la suprafață imediat ce ai crește K. Fără filtru, fără semnal de calitate dincolo de support.

v1 — input shaping

Cinci pași de preprocesare aplicați **înainte** de mining:

1. **Prefix-rank tie-break sort.** P4 are `M:metformin` și `C:diabetes` la aceeași dată; v0 sortează alfabetic (M înainte de C), deci P4 = `[1,0,2]`. v1 forțează ordinea C → M → P → CP, deci P4 devine `[0,1,2]` (`C:diabetes, M:metformin, P:foot_exam`). Acum P4 se potrivește cu același pattern ca P1/P2/P3 în loc să arate ca un outlier. *Efect pe exemplu:* `[0,1]` (`C:diabetes, M:metformin`) devine sup=4 curat, nu împărțit între două aranjamente.

2. **Item blacklist.** `medication_review_due` e în blacklist. P3 devine `[0,1]` (`C:diabetes, M:metformin`), P5 devine `[5,6,1]` (`C:asthma, M:albuterol, M:metformin`). Pattern-ul `M:metformin, M:medication_review_due` (sup=2) dispare — exact genul de noise pe care nu-l vrem poluând top-K.

3. **Time window.** Nu se vede pe exemplul de jucărie (toate datele sunt în window). Pe date reale, asta elimină evenimente vechi de un deceniu care nu reflectă traiectoria curentă a pacientului.

4. **Consecutive-duplicate collapse.** Nu se vede aici. Pe date reale, asta elimină run-uri `M:lisinopril → M:lisinopril → M:lisinopril` din evenimente de refill, care altfel ar umfla support-ul pentru repetiție trivială.

5. **Per-patient length cap.** Nu se vede (max length = 4). Pe date reale, capping la 25 de evenimente menține adâncimea recursiei PrefixSpan mărginită; altfel un singur pacient cu istoric lung poate domina runtime-ul.

După v1, database-ul (encodat) e:

```
P1  [0, 1, 2, 3]        C:diabetes, M:metformin, P:foot_exam, C:hypertension
P2  [0, 1, 2]           C:diabetes, M:metformin, P:foot_exam
P3  [0, 1]              C:diabetes, M:metformin
```

```
P4  [0, 1, 2]          C:diabetes, M:metformin, P:foot_exam
P5  [5, 6, 1]          C:asthma, M:albuterol, M:metformin
```

v2 — mining configuration

Două schimbări:

1. **`closed=True`.**

   Definiție. Un frequent pattern `P` e **closed** dacă și numai dacă niciun proper superset al lui `P` nu are același support:

   ```
   P e closed  ⟺  ∀ P' ⊋ P :  support(P') < support(P)
   ```

   De ce vrem closed patterns. Dacă `[0]` (`C:diabetes`) și `[0,1]` (`C:diabetes, M:metformin`) au amândouă support 4, fiecare pacient care suportă pattern-ul mai scurt îl suportă și pe cel mai lung — pattern-ul mai lung poartă strict mai multă informație. Raportându-le pe amândouă, irosești un slot din top-K și aglomerezi output-ul. Closedness colapsează fiecare „tier de support egal" la cel mai lung reprezentant al lui.

   Closed vs maximal. Un pattern e *maximal* dacă niciun superset nu e frequent deloc (orice superset e sub `minsup`). Cele două condiții se leagă astfel:

   ```
   maximal  ⟹  closed       (orice maximal pattern e closed)
   closed   ⟹  maximal       (closed e mai slab)
   ```

   Folosim closed, nu maximal, pentru că maximal prunează prea mult. Exemplu: dacă `[0,1]:4` are un length-3 superset la support 2 (exact peste `minsup`), maximal ar arunca complet pattern-ul length-2 — pierzând informația de support mult mai mare. Closed colapsează patterns doar *la același nivel de support*, care e exact redundanța pe care chiar vrem s-o eliminăm.

   Pe exemplul nostru. `[0]:4` și `[0,1]:4` au support identic. Cel mai scurt e dominat și nu e closed; cel mai lung e closed. După pruning: `[0,1]:4` supraviețuiește, `[0]:4` nu. Nu irosim un slot din top-K pe prefix-ul redundant.

2. **Overmine 20× apoi ia top-K.** Minăm `K × 20 = 60` patterns într-un pool și alegem din el. Pe un exemplu de 5 sequences asta e irelevant (ai epuiza lattice-ul). Pe date reale e critic: fără overmining, closed-by-itself top-K se umple deja cu high-support trivia (ex. toate dental patterns) și scoring-ul/diversificarea de mai jos nu mai au cu ce lucra.

v3 — ranking

Acum punctăm patterns. Metrica e **geometric mean lift (GML)**, cu un type-diversity bonus deasupra. Secțiunea explică întâi lift-ul, apoi cum generalizăm la k items, apoi de ce geometric mean și forma logaritmică, apoi numerele lucrate pe exemplu.

3a. Lift — fundalul de association-rule

Pentru două items A și B, **lift** e măsura standard de association din market-basket:

$$\text{lift}(A, B) = P(A \wedge B) \,/\, (P(A) \cdot P(B))$$

Dacă A și B ar fi independente, $P(A \wedge B) = P(A) \cdot P(B)$ și lift = 1. Lift > 1 înseamnă că A și B co-apar *mai des decât ar prezice independența* (asociere pozitivă). Lift < 1 înseamnă că se evită activ. Lift = 1 înseamnă fără asociere.

Plain support nu poate distinge „două items comune co-apărând cum era de așteptat" de „două items rare co-apărând surprinzător de des". Lift poate.

3b. Generalizarea lift la k items: de ce geometric mean

Generalizarea naturală de la 2 items la un pattern $P = [\text{item}_1, \ldots, \text{item}_k]$ e:

$$\text{lift}_k(P) = P(P) \,/\, \prod_i P(\text{item}_i)$$

cu $P(\text{item}_i) = \text{single_support}[\text{item}_i] \,/\, n$ și $P(P) = \text{support}(P) \,/\, n$. Formula merge, dar două probleme o fac inutilizabilă ca atare:

1. **Underflow.** Pe măsură ce k crește, $\prod P(\text{item}_i)$ e un produs de probabilități mici. Cu k = 6 și probabilități per-item în jur de 0.01, produsul e 10^{-12}. Float64 îl gestionează, dar precizia se degradează și comparațiile devin instabile.
2. **Incomparabilitate între length-uri diferite.** Un length-3 pattern și un length-6 pattern au numitori de scară foarte diferită (unul e produs triplu, celălalt produs de șase ori). Ranking-ul patterns de length-uri diferite unul față de altul devine fără sens.

Ambele probleme dispar dacă luăm **rădăcina de ordin k** a numitorului — adică înlocuim produsul cu geometric mean-ul probabilităților per-item:

$$\text{gml}(P) = P(P) \,/\, \text{geomean}(P(\text{item}_1), \ldots, P(\text{item}_k))$$

$$= \frac{\text{support}(P) \,/\, n}{(\prod_i \text{single_support}[\text{item}_i] \,/\, n)^{(1/k)}}$$

Acum e un scor normalizat per-item: răspunde la „cu cât mai des apare acest pattern decât ar prezice frecvențele de bază ale items lui, *mediat peste items*". Patterns de length-uri diferite sunt direct comparabile.

De ce geometric mean și nu aritmetic. Media aritmetică a probabilităților e dominată de cea mai mare valoare — deci un pattern care conține un item foarte comun ar părea „comun per total". Geometric mean e dominat de cea mai *mică* valoare — un pattern care conține un item rar e considerat „rar per total", ceea ce e ce vrem.

E aceeași logică care face F-score (harmonic mean) să prefere precision și recall echilibrate față de media aritmetică; vrei ca agregatul să reflecte veriga cea mai slabă, nu cea mai puternică.

De ce forma logaritmică în cod. Calculul direct al $(x_1 \cdot x_2 \cdot \ldots \cdot x_k)^{(1/k)}$ face underflow pentru x_i mici. Identitatea

$$\log(\text{geomean}(x_i)) = (1/k) \cdot \Sigma_i \log(x_i) = \text{mean}(\log(x_i))$$

ne lasă să lucrăm integral în log-space. De aici codul efectiv:

```
log_n = np.log(n_patients)
log_item_freqs = np.log(np.maximum(single_support[unique_items], 1)) - log_n
log_gml = np.log(sup) - log_n - np.mean(log_item_freqs)
gml = np.exp(log_gml)
```

`np.maximum(..., 1)` protejează contra `log(0)` dacă vreun item are cumva count zero.

3c. Exemplu lucrat

După preprocesarea din v1, single-item supports peste cei 5 pacienți sunt:

```
C:diabetes      4
M:metformin     5
P:foot_exam     3
C:hypertension  1
C:asthma        1
M:albuterol     1
```

`n = 5`, `log(5) ≈ 1.609`.

Pattern `[0, 1]` (`C:diabetes, M:metformin`) (sup = 4, k = 2):

```
log P(item) values:  log(4)−log(5) = log(0.8) ≈ −0.223
                     log(5)−log(5) = log(1.0) =   0.000
mean             :   −0.112

log_gml = log(4) − log(5) − (−0.112)
        = log(0.8) + 0.112
        = −0.223 + 0.112
        = −0.111

gml ≈ exp(−0.111) ≈ 0.895
```

Pattern `[0, 1, 2]` (`C:diabetes, M:metformin, P:foot_exam`) (sup = 3, k = 3):

```
log P(item) values:  log(4)−log(5) ≈ −0.223
                     log(5)−log(5) =   0.000
                     log(3)−log(5) ≈ −0.511
mean             :  −0.245

log_gml = log(3) − log(5) − (−0.245)
        = log(0.6) + 0.245
        = −0.511 + 0.245
        = −0.266

gml ≈ exp(−0.266) ≈ 0.766
```

Observă că GML-ul pattern-ului de 2 items (0.895) e *mai mare* decât al celui de 3 items (0.766) pe acest exemplu, pentru că toate cele trei items sunt foarte frequent în lumea noastră de 5 pacienți. Pe date reale cu 3000 de items în vocabular unde majoritatea au single-support în jur de 0.01–0.05, un pattern co-apărând de 3 items ar copleși orice pattern de 2 items. Exemplul e prea mic ca să facă vizibil câștigul GML — dar formula e aceeași.

Type-diversity bonus-ul, definit în continuare, *va* reordona exemplul în modul așteptat.

3d. Type-diversity bonus

Bonus-ul înmulțește GML cu:

```
gml_final(P) = gml(P) · ( 1 + 0.3 · (n_types(P) − 1) )
```

unde `n_types(P)` e numărul de type prefixes distincte (C, M, P, CP) din pattern. Multiplicatori concreți:

```
1 type   → ×1.0
2 types  → ×1.3
3 types  → ×1.6
4 types  → ×1.9
```

Pe exemplu:

```
[0, 1]     (C:diabetes, M:metformin)              : 2 types {C,M}   → 0.895 × 1.3
= 1.164
[0, 1, 2]  (C:diabetes, M:metformin, P:foot_exam) : 3 types {C,M,P} → 0.766 × 1.6
= 1.226
```

Acum pattern-ul de 3 items câștigă (1.226 > 1.164) — bonus-ul are destulă forță cât să răstoarne ranking-ul pe un exemplu unde raw GML nu o făcea.

Asumpția de domeniu codificată aici. Un pattern care traversează condition → medication → procedure spune o poveste clinică mai bogată decât trei medicamente la rând. Al doilea e adesea un refill schedule sau o enumerare de medication-class; primul e o *treatment trajectory* (un diagnostic, un răspuns, o intervenție). Recompensând type diversity, scoatem la suprafață patterns în formă de trajectory în top-K.

De ce 0.3 anume. Hyperparameter, tunat empiric. La 0.1 bonus-ul era prea slab ca să reordoneze ceva vizibil în top-100. La 0.5 bonus-ul copleșea complet support-ul și combinații exotice de 4-type băteau patterns reale de co-occurrence. 0.3 a fost punctul în care top-100 se îmbunătățea vizibil fără s-o ia razna. Nu există optim teoretic; e un knob.

v4 — filtering și diversification

Șase verificări independente aplicate în timp ce alegem top-K final din pool-ul GML-ranked. Definim fiecare, o justificăm și (unde exemplul poate arăta) lucrăm numerele.

4a. Run-of-3-same-type filter

Regula. Elimină orice pattern care conține trei items *consecutive* de același type prefix.

De ce. Trei C, M sau P consecutive sunt aproape mereu artefacte, nu povești clinice:

- `M:lisinopril, M:lisinopril_refill, M:lisinopril_renewal` — birocrație de refill
- `P:dental_exam, P:fluoride_application, P:supragingival_scaling` — protocolul dentar Synthea, emis verbatim per pacient
- `C:fever, C:cough, C:fatigue` — codare de simptome concurente, nu progresie temporală

Astea inundă pool-ul când GML e mare (mai multe items co-apărând) dar nu spun nicio poveste cauzală. Eliminarea lor e ieftină și eficientă.

Implementare. Glisezi o fereastră de length-3 peste array-ul de type al pattern-ului (în întregi); elimini dacă cele trei intrări ale vreunei ferestre sunt egale. O(k) per pattern, vectorizat cu `numpy` în practică.

Pe exemplu. Nu există run-uri length-3 same-type după preprocesarea din v1. Filtrul nu se declanșează niciodată.

4b. Explicit closedness pass

Regula. Sortează patterns supraviețuitoare după support. Pentru fiecare P, verifică dacă vreun P' cu `support(P') = support(P)` e un strict superset al lui P cu P o subsequence a lui P'. Dacă da, elimină P.

De ce un al doilea pass după `closed=True`. `closed=True` din bibliotecă prunează local în timpul recursiei PrefixSpan (§3). E corect *în interiorul arborelui de recursie*, dar pool-ul din v4 combină patterns din branch-uri de recursie diferite și le reordonează după GML. Perechi redundante scapă. Pass-ul explicit e o măturare belt-and-braces care prinde ce a ratat verificarea locală. Pe date reale, elimină 5–15% din pool-ul GML-ranked.

Implementare. Grupare după support (`sup_to_idx: dict[int, list[int]]`); pentru fiecare pattern verifică doar same-support candidates (alte support-uri nu pot domina pentru că dominatorul trebuie să aibă support *egal*); folosește `frozenset.issubset` ca pre-filtru ieftin înainte de verificarea de subsequence mai scumpă `_is_int_subseq`.

4c. Jaccard dedup

Definiție. Pentru două mulțimi finite A și B:

```
J(A, B) = |A ∩ B| / |A ∪ B|
```

Interval [0, 1]: 0 = disjuncte, 1 = identice. Calculăm Jaccard pe *item sets*-urile a două patterns (adică mulțimea de items distincte din fiecare), deci ignoră ordinea și multiplicitatea. Două patterns `[a, b, c]` și `[c, b, a]` primesc J = 1.0 aici, deși sunt sequences diferite.

Regula. Parcurge pool-ul GML-ranked de sus în jos. Pentru fiecare candidate, calculează J față de fiecare pattern deja-ales. Sari peste candidate dacă max J ≥ 0.80.

De ce item-set Jaccard, nu sequence similarity. Pentru clinical pattern discovery, *bag of clinical events* dintr-un pattern poartă majoritatea înțelesului. Două patterns cu aceleași items în ordine diferită sunt de obicei aceeași constatare clinică (ordinea din sequence reflectă când au fost înregistrate evenimentele, ceea ce nu e mereu o ordine cauzală). Măsuri de sequence-similarity (bazate pe LCS, edit distance) ar lăsa near-duplicates să treacă.

De ce Jaccard și nu Dice sau cosine.

- **Dice** e `2|A∩B| / (|A|+|B|)`. Algebric înrudit: `Dice = 2J / (1+J)`. La Dice threshold 0.80, Jaccard-ul implicat e doar `0.80 / (2 − 0.80) = 0.67` — mai laxă. Am încercat-o; near-duplicates vizibile au scăpat.
- **Cosine** pe item-presence vectors e `|A∩B| / √(|A|·|B|)`. *Nu* penalizează size mismatch. Exemplu: A de size 3 conținut complet în B de size 10 dă `cos = 3 / √30 ≈ 0.55`, ceea ce sună moderat — dar item set-ul lui A e 100% acoperit de B și am vrea o măsură de similaritate care semnalează asta. Jaccard întoarce corect `3 / 10 = 0.30`, reflectând că B poartă 7 items pe care A nu le are.
- **Jaccard** e simetric, mărginit [0, 1], penalizează size mismatch și are citirea umană clară „fracțiunea de items împărtășite". 0.80 înseamnă „împart 80% din items".

De ce threshold 0.80. Empiric. La 0.70 dedup-ul era destul de agresiv cât să elimine patterns legitim distincte care împart 3/5 items. La 0.90 near-duplicates cu un item substituit (ex. `P:foot_exam` vs `P:hba1c_test`) scăpau. 0.80 — „împart patru din cinci items" — a fost punctul dulce.

Exemple lucrate. Fie A pattern-ul deja-ales; pentru fiecare candidate calculează J(A, candidate).

```
A = {C:diabetes, M:metformin, P:foot_exam}                        (3 items)

candidate 1 = {C:diabetes, M:metformin, P:hba1c_test}             (3 items)
  intersection = {C:diabetes, M:metformin}                         size 2
  union        = {C:diabetes, M:metformin, P:foot_exam,
                  P:hba1c_test}                                    size 4
  J = 2 / 4 = 0.50                                  → păstrat (procedure diferită)

candidate 2 = {C:diabetes, M:metformin, P:foot_exam, C:hypertension}  (4 items)
  intersection = A                                                 size 3
  union        = candidate 2                                       size 4
  J = 3 / 4 = 0.75                                  → păstrat (adaugă info de
hypertension)

candidate 3 = {C:diabetes, M:metformin, P:foot_exam, P:hba1c_test}    (4 items)
```

```
  J = 3 / 4 = 0.75                                  → păstrat (adaugă info de
hba1c)

candidate 4 = {C:diabetes, M:metformin, P:foot_exam, P:hba1c_test, C:hypertension}
(5 items)
  intersection = A                                              size 3
  union        = candidate 4                                     size 5
  J = 3 / 5 = 0.60                                  → păstrat

candidate 5 = {C:diabetes, M:metformin, P:foot_exam, C:hypertension,
C:hyperlipidemia} (5 items)
  J = 3 / 5 = 0.60                                  → păstrat

candidate 6 = {M:metformin, P:foot_exam, C:diabetes}            (3 items,
reordonat)
  intersection = A                                              size 3
  union        = A                                              size 3
  J = 3 / 3 = 1.00                                  → ELIMINAT  (același item-set)

candidate 7 = {C:diabetes, M:metformin, P:foot_exam, M:atorvastatin}  (4 items)
  J = 3 / 4 = 0.75                                  → păstrat

candidate 8 = un pattern care împarte 4/5 items cu un pattern de 5 items ales:
  J = 4 / 5 = 0.80                                  → ELIMINAT  (exact la
threshold)
```

Asimetria se vede: un pattern care *adaugă* un singur item la unul ales mai mic supraviețuiește aproape mereu (J ≤ 0.75 pentru adăugare la size-3 sau size-4). Un pattern care *reordonează* aceleași items moare mereu (J = 1.0). Un pattern care adaugă un item la un pattern ales de *5 items* atinge threshold-ul și moare. Mușcătura regulii e exact: „nu păstra două patterns care împart ≥ 4/5 din items".

4d. Per-prefix cap (MAX_PER_PREFIX = 6)

Regula. În timp ce alegi top-K, urmărește câte patterns alese încep cu fiecare item. Sari peste orice candidate al cărui prim item are deja 6 patterns alese.

De ce. Pe date reale ~40% din pacienți sunt diabetici, deci patterns `C:diabetes, …` domină ranking-ul GML. Fără un cap, top-100 ajungea cu ~35 de patterns care încep toate cu `C:diabetes`. Cap-ul forțează algoritmul să scoată la suprafață patterns din alte leading conditions, medications și procedures.

De ce 6. Cu `TOP_K = 500` și de obicei 50–80 de leading items distincte în pool, `6 × 80 = 480` e aproape de TOP_K — cap-ul e binding în practică fără să lase slot-uri goale. La 3 top-K era sub-umplut; la 10 inundația de diabet era încă vizibilă.

4e. Per-suffix cap (MAX_PER_SUFFIX = 20)

Regula. Aceeași idee pe *ultimul* item al fiecărui pattern. Cap mai lax (20 vs 6).

De ce mai lax. Multe chain-uri clinice legitime se termină legitim la aceeași procedure sau condition: `P:foot_exam`, `C:hypertension`, `M:atorvastatin` sunt endpoint-uri comune atinse din sequences de lead foarte diferite. Suffix capping agresiv ar omorî patterns genuin distincte. 20 limitează doar inundația flagrantă.

4f. Dental cap (MAX_DENTAL_PATTERNS = 15)

Regula. Un pattern e „dental-majority" dacă ≥ 50% din items lui *distincte* se potrivesc cu vreun DENTAL_KEYWORD (`dental`, `fluoride`, `gingiv`, `supragingival`, `subgingival`, `oral health`, `tooth`, `teeth`). Cel mult 15 dental-majority patterns pot intra în top-K.

De ce e nevoie. Synthea (generatorul de date) emite un pattern standardizat de fluoride-protocol pentru aproape fiecare pacient:

```
<P:dental_exam, P:supragingival_scaling, P:fluoride_application,
 P:dental_exam, ...>
```

Acest protocol are support astronomic și GML mare (mai multe items co-apărând). Chiar și după Jaccard dedup, generează 30–50 de ordonări diferite și variații de items care trec threshold-ul 0.80. Fără un cap dur, top-100 era ~40% dentar și inutil clinic.

De ce un cap și nu un blacklist. Evenimentele dentare *sunt* uneori semnificative clinic (infecții orale precedând evenimente cardiace, de exemplu). Nu vrem să eliminăm categoria. 15 slot-uri sunt destule cât să scoatem la suprafață constatări dentare reale prevenind în același timp inundația.

Dental cap-ul e *singurul* filtru din acest pipeline care hardcodează domain knowledge. Celelalte (run-of-3, closedness, Jaccard, prefix/suffix caps) sunt domain-agnostic. Dacă ai reutiliza acest pipeline pe date de sequence non-clinice, dental cap-ul e primul lucru de scos.

Ce produce exemplul după v4

După ce blacklist-ul din v1 a eliminat `medication_review_due`, closed pool-ul are doar:

```
[0, 1]     (C:diabetes, M:metformin)              sup=4   gml ≈ 1.16
[0, 1, 2]  (C:diabetes, M:metformin, P:foot_exam) sup=3   gml ≈ 1.23
```

Niciunul nu declanșează vreun filtru din v4 (fără run-of-3, fără pereche de near-duplicate de Jaccard-deduplicat, fără prefix/suffix flood, fără dental-majority). Amândouă supraviețuiesc.

Asta e *corect*: exemplul e prea mic ca să se declanșeze vreun filtru din v4, exact de aceea nu i-am vedea niciodată câștigul pe el. Pe date reale (~1000 pacienți, vocab ~3000), diferența dintre top-100 din v3 și top-100 din v4 e dramatică — v3 e dominat de near-duplicates și un singur motiv; v4 e larg, citibil și semnificativ clinic.

v5 — downstream orchestration

Trei preocupări, toate *după* ce patterns sunt alese:

1. **`attribute_patients`.** Pentru fiecare pattern păstrat, găsește mulțimea de pacienți a căror sequence chiar îl conține. Naiv: scanezi fiecare patient sequence per pattern (O(K × N × L)). v5 construiește un inverted index `item → [patient_ids]`, apoi pentru fiecare pattern intersectează posting lists-urile celor mai rare items ale lui întâi, apoi verifică subsequence containment pe intersecția mică. *De ce*

contează: frontend-ul trebuie să arate „cărui pacient i se aplică acest pattern" pentru fiecare clustered chart, iar scan-ul per-pattern e prea lent.

2. **Per-cluster mining mode (`--n-clusters N`).** Rulează tot pipeline-ul o dată per cluster label (0..N−1 plus -1 pentru outliers), restricționând `build_sequences` la patient IDs-urile acelui cluster. Patterns per-cluster sunt mult mai interesante clinic decât cele globale, pentru că fiecare cluster are un fenotip coerent.

3. **Persistence (`save_prefix_span_results`).** Scrie patterns în `prefix_span_pattern` și membership-urile în `prefix_span_patient_pattern`, cu cluster label-ul ca foreign key. Plus un dump `patterns_*.txt` pentru inspecție umană în timpul dezvoltării.

---

5. Rezumat: ce îți dă PrefixSpan clasic și ce am adăugat noi

Biblioteca îți dă algoritmul clasic (cu pseudo-projection și un top-K heap). Tot ce e în v1–v5 e engineering specific proiectului, peste o implementare curată a lucrării din 2001.

Cele 17 îmbunătățiri se împart curat în cinci preocupări:

Layer	Ce repară	Îmbunătățiri
v1 input shaping	sequences sunt noisy și nemărginite	sort tie-break, blacklist, time window, dup collapse, length cap
v2 mining config	raw top-K e dominat de trivia	overmine ×20, `closed=True`
v3 ranking	plain support recompensează items comune	GML, type-diversity bonus
v4 filtering	top-K păstrat e plin de near-duplicates și inundații dintr-un singur domeniu	run-of-3, explicit closedness, Jaccard, prefix/suffix/dental caps
v5 orchestration	patterns trebuie să ajungă în aplicație per-cluster	attribution, per-cluster mode, persistence

Dacă poți răspunde la „ce se schimbă între vN și vN+1, și de ce schimbarea aia e vizibilă pe date reale dar nu pe un exemplu de 5 sequences" pentru fiecare pas, înțelegi pipeline-ul destul de bine cât să-l aperi.